

Learning Tanstack Query

I was doing a project and I stumbled upon this. And got some nice reviews from the community too. So, here I go.

My name is `Md. Rishat Talukder` and this is `Learning Tanstack Query with ITVAYA`

- [LinkedIn](#)
- [YouTube](#)
- [github](#)
- [Gmail](#)
- [discord](#)

Prerequisites

- Must have nodejs and npm installed in your system.
- Good understanding of React and basic request handling with Axios and state management with contextAPI and useReducer.
- Good knowledge of JavaScript.
- Basic knowledge of TypeScript because I'll be using typescript for this repo.(For now just think of everything is javascript but with .ts or .tsx instead of .js or .jsx)

Setup

You can install everything in this project using `npm` but I don't like `npm` so I'm using `pnpm` instead.

Install pnpm

You can follow the [official installation guide](#) to install pnpm for you operating system.

Or if you have nodejs installed, you can use the following command to install pnpm globally.

Just open your terminal and run the following command:

```
npm install -g pnpm@latest-11
```

Wait for the installation to finish and you're good to go.

Make a new ReactTS project using Vite

```
pnpm create vite
```

After running this command you'll be prompted to enter the name of your project.

I entered `code`. That's why you'll see a folder named `code` in the [repository](#). It's the react app.

You can name the app whatever you like.

Then you'll be asked to select a framework. Select `react`.

then you have to choose a variant. If you are a pure js programmer choose `Javascript`. I'll choose `Typescript` for this repo because I need some practice of typescript.

Don't worry. Everything is the same just I'll be adding some extra colons to just tell what type of variable it is.

Now, run the following commands:

```
cd code
pnpm install
pnpm dev
```

`pnpm dev` will start the server and open the app in your browser by typing `o + enter` in your terminal.

Install dependencies

Now, we will need some extra libraries for tanstack to work properly.

- Axios for making HTTP requests
- Tanstack Query because this article is all about tanstack query.

Now, to install tanstack query and axios, run the following command:

```
pnpm add @tanstack/react-query axios
```

Also, if you are using `vs code` you should install the `eslint plugin`.

```
pnpm add -D @tanstack/eslint-plugin-query
```

One last thing, a temporary backend server. We can do that using `json-server`.

```
pnpm add json-server
```

Now, make a new file named `db.json` in your `root` project folder. It's a json file that will hold some temporary data.

```
{
  "users": [
    {
      "id": 1,
      "name": "John Doe",
      "email": "2B7Pd@example.com",
      "password": "password123"
    }
  ]
}
```

As, you can see that here I have a `users` property inside the `db.json` file and it has an array of users. The properties inside the json will become the endpoint for that data.

Run this command:

```
npx json-server db.json
```

This will start the server and you can access it by typing `http://localhost:3000` in your browser.

Here, if you want to access the users array you have to add `/users` to the end of the url. So, the url will be `http://localhost:3000/users`.

Try it out, you'll see a list of users in the browser.

Integrate tanstack query in the project

First things first, after all the installations are done, open the `src` folder of your project and remove all the files except `App.jsx/App.js/App.ts/App.tsx` and `main.jsx/main.js/main.ts/main.tsx` files.

Now, you'll see a lot of errors. No need to panic. You'll see those errors because all the files we just removed are being imported in the `main` file or the `App` file. Just removing those bad imports should fix the issues.

From now on think of every `.ts` as `.js` and every `.tsx` as `.jsx` if you made a `reactjs`` app.

Your `main.jsx/main.js/main.ts/main.tsx` file should look something like this:

```
1  | //src/main.tsx
2  | import { StrictMode } from "react";
3  | import { createRoot } from "react-dom/client";
4  | import App from "./App.tsx";
5  |
6  | createRoot(document.getElementById("root")!).render(
7  |   <StrictMode>
8  |     <App />
9  |   </StrictMode>,
10 | );
```

Your `App.tsx` file should look something like this:

```
1  | //src/App.tsx
2  | function App() {
3  |   return <div>Nice and clean</div>;
4  | }
5  |
6  | export default App;
```

Now, we have a clean app and we should now integrate tanstack query in it.

First, I would recommend you go to the [documentations](#). Read the documentation from `overview` to `Devtools`.

Reading docs is an important skill for any developer.

First we have to wrap out app component with the `QueryClientProvider` component.

```

1  //src/main.tsx
2  import { StrictMode } from "react";
3  import { createRoot } from "react-dom/client";
4  import App from "./App.tsx";
5  import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
6
7  const queryClient = new QueryClient();
8
9  createRoot(document.getElementById("root")!).render(
10    <StrictMode>
11      <QueryClientProvider client={queryClient}>
12        <App />
13      </QueryClientProvider>
14    </StrictMode>,
15  );

```

Now, let's break it down.

- First, we import the `QueryClient` and `QueryClientProvider` from `@tanstack/react-query`.
- Then, we create a new instance of `QueryClient`. This query client object will hold the much needed information for tanstack query to work.
- Now, we pass this `queryClient` object to the `QueryClientProvider` component. This component will act as a context provider for all the data and making tanstack query work everywhere in the whole app.
- Then, we wrap our `App` component in the `QueryClientProvider` component. This will make tanstack query work in our app.

Optional but recommended: Adding devtools

Tanstack query comes with a built-in devtools. And you can use this dev tools to monitor if every query is working properly in your app and also you can keep track of the state variables controlled by tanstack query.

run this following command in terminal:

```
pnpm add @tanstack/react-query-devtools
```

Now, open the `main.tsx` file and import the `ReactQueryDevtools` component from `@tanstack/react-query-devtools` and right above your `App` component render the `ReactQueryDevtools` component.

```

1  //src/main.tsx
2  ...
3
4  import { ReactQueryDevtools } from "@tanstack/react-query-devtools";
5
6  const queryClient = new QueryClient();
7
8  createRoot(document.getElementById("root")!).render(
9    <StrictMode>
10     <QueryClientProvider client={queryClient}>
11       { /* Right above your App */ }
12       <ReactQueryDevtools/>
13       <App />
14     </QueryClientProvider>
15   </StrictMode>,
16 );

```

YOU restart the server and after opening it in the browser you should see a floating icon on the bottom right corner of your screen.

When you click it it'll automatically open a tab section where you can see a lot of information about your queries.

ANND THE SETUP IS DONE. WELL, THAT TOOK LONGER THAN I EXPECTED. 🤔

Now, let's get into the action.

Introduction to Tanstack Query

What is it?

Tanstack query (Formally known as react query) is like a global state manager for data fetching and caching.

You know that when you are fetching data from the server you have to make a request and wait for the response and there can be different possible situations, mainly:

- Loading
- Success
- Error

Then you have to set it up a side-effect using `useEffect` hook to activate the fetching logic when a component is starting.

This is generally how things goes.

Well, `tanstack query` does it all for you and many more. It'll handle all the needed states and give extra features that can be a little tougher to implement by yourself.

Tanstack query has three core concepts:

- Queries
- Mutations
- Query Invalidation

How to use it?

First let's fetch the data using the traditional methods.

I'll make a `api` configuration file so that I don't have to write the same code again and again.

Make a new file named `api.ts` in your `src` folder.

```
1  //src/api.ts
2  import axios from "axios";
3
4  export const api = axios.create({
5    baseURL: "http://localhost:3000/",
6  });
```

Now, I can use this `api` object to send requests to the json server at `http://localhost:3000/`. I won't have to write the same code again and again.

Now, let's get the data from the `/users` endpoint and render it in the app.

```
1 //src/App.tsx
2 import { useEffect, useState } from "react";
3 import { api } from "../api";
4
5 function App() {
6   // types for the data
7   type dataType = {
8     id: number;
9     name: string;
10    email: string;
11    password: string;
12  };
13
14  // states I have to use
15  const [data, setData] = useState<dataType[] | null>(null);
16  const [loading, setLoading] = useState(false);
17  const [error, setError] = useState<Error | null>(null);
18
19  // function to fetch the data
20  const fetchData = async () => {
21    try {
22      setLoading(true);
23      const res = await api.get<dataType[]>("/users");
24      setData(res.data);
25    } catch (error) {
26      setError(error as Error);
27    } finally {
28      setLoading(false);
29    }
30  };
31
32  // side-effect to fetch the data when loading
33  useEffect(() => {
34    fetchData();
35  }, []);
36
37  // checking the states
38  if (loading) return <div>Loading...</div>;
39
40  if (error) return <div>Error: {error.message}</div>;
41
42  if (!data) return <div>No data</div>;
43
44  //finally render the data
45  return (
46    <div>
```

```

47     {data.map((user) => (
48         <div key={user.id}>
49             <h2>{user.name}</h2>
50             <p>{user.email}</p>
51         </div>
52     ) ) }
53 </div>
54 );
55 }
56
57 export default App;

```

This is what a general data fetching logic looks like. 4-5 steps each time. If you are a little experienced with react, you already know whats going on here.

1. Set the states for loading, error and data.
2. Create a function to fetch the data that will set the states to loading to true and try to fetch the data from the api. If successful, set the data state to the response data. If not, set the error state to the error. Finally, set the loading state to false.
3. Check every state and return the appropriate component.

That's how it normally works. Now, let's see how to use tanstack query.

```
1 //src/App.tsx
2 import { useEffect, useState } from "react";
3 import { api } from "../api";
4 import { useQuery } from "@tanstack/react-query";
5
6 function App() {
7   // types for the data
8   type dataType = {
9     id: number;
10    name: string;
11    email: string;
12    password: string;
13  };
14
15  // using tanstack query to get all the previous states directly
16  const { data, error, isLoading } = useQuery({
17    queryKey: ["users"],
18    queryFn: async () => {
19      const res = await api.get<dataType[]>("/users");
20      return res.data;
21    },
22  });
23
24  // checking the states
25  if (isLoading) return <div>Loading...</div>;
26
27  if (error) return <div>Error: {error.message}</div>;
28
29  if (!data) return <div>No data</div>;
30
31  //finally return the data
32  return (
33    <div>
34      {data.map((user) => (
35        <div key={user.id}>
36          <h2>{user.name}</h2>
37          <p>{user.email}</p>
38        </div>
39      ))}
40    </div>
41  );
42 }
43
44 export default App;
```

This is the same logic as before but using tanstack query. Just one hook doing all the heavy lifting. Now, the main part is that I removed the `useState` and `useEffect` hooks completely and the all the needed states are given to me directly from tanstack query.

PS: in tanstack query the loading state is called `isPending`.

now, let's break it down.

The Query Object

Let's analyze the `useQuery` hook.

```
1  //src/App.tsx
2  const { data, error, isPending } = useQuery({
3    queryKey: ["users"],
4    queryFn: async () => {
5      const res = await api.get<dataType[]>("/users");
6      return res.data;
7    },
8  });
```

The `useQuery` hook takes an object as a parameter and inside that object you mainly have to pass 2 things.

- The `queryKey` which is an array of strings that will be used to identify the query.
- The `queryFn` which is a function that will be used to fetch the data.

Let's talk about what the `useQuery` hook returns.

What happens when you use the `useQuery` hook?

First the `useQuery` runs the `queryFn` and if successful it returns what is returned from the `queryFn` to the `data` state.

That's why I returned `res.data` from the `queryFn`. It's exactly like the try block in the `fetchData` function.

`useQuery` hook will automatically set the `isPending` state to true while the `queryFn` is running and will set it to false once the `queryFn` is done.

And if the `queryFn` throws an error, it will set the `error` state to the error.

Exactly what we manually did with manual use of `useState` and `useEffect`.

So, the `useQuery` hook will return a `useQueryResult` object that contains the `data`, `error` and `isPending` states.

And this object can have a looot of other properties that can be used to control the query.

Just take a look at the [documentation](#).

This query object will have attributes like `data`, `isPending`, `error`, `fetchStatus`, `isFetching` and many more.

You can use these states and function for your specific needs.

And also this object is returned by other query hooks like `useQueries`, `useQueryClient` or `useSuspenseQuery`.

What does the `queryKey` do?

The `queryKey` is an array of strings that will be used to identify the query. It's important to use a unique key for each query.

Let's make a button to fetch the data again when we click on it.

```
1  //src/App.tsx
2  ...
3
4  function App() {
5    ...
6
7    const { data, error, isPending, refetch } = useQuery({
8      queryKey: ["users"],
9      queryFn: async () => {
10        const res = await api.get<dataType[]>("/users");
11        return res.data;
12      },
13    });
14
15    ...
```

Here, I'm using the `refetch` function from the `useQueryResult` object. This function will refetch the data.

Now we, can just add button to trigger the `refetch` function.

```
1  //src/App.tsx
2  ...
3
4  //finally return the data
5  return (
6    <div>
7      ...
8      <button onClick={() => refetch()}>Re-fetch</button>
9    </div>
10  );
11  ...
```

And that's it!

Now, go open the frontend in your browser and click the re-fetch button. You'll see that nothing is being re-fetched.

This because of the `queryKey`. The `queryKey` is used to identify the query and as we are re-fetching the same query, it instantly displays the existing cached data so the UI remains interactive. If the newly fetched data has changed, TanStack Query automatically overwrites the cache and updates the React state to reflect the new data on the screen. If the fetched data is identical, it leaves the cache as-is and optimizes performance by preventing an unnecessary UI re-render

This is awesome right? The `useQuery` hook is doing almost all of the heavy lifting for us.

So, `queryKey` property is a unique key that will be used to identify the query. It's important to use a unique key for each query.

Now, if you really want to see if the re-fetch is happening or not you can destructure the `isRefetching` state from the `query` object and check if it's true or false.

This state is set to true when the query is being refetched.

Try it out yourself.

We'll get to know more properties of this object as we progress through this article. But time to go deeper into the `useQuery` hook.

Query Options

I'll restructure some things before we get going.

It's best practice to not write the query function directly in the `useQuery` hook. Instead, write it in a separate function and pass it as a prop to the `useQuery` hook.

I'll put the query from the first example in the `fetchUsers` function and pass it as a prop to the `useQuery` hook.

Make a new file named `queryFunctions.ts` in your `src` folder.

```
1  //src/queryFunctions.ts
2  import { api } from "../api";
3
4  export const fetchUsers = async () => {
5    const res = await api.get("/users");
6    return res.data;
7  };
```

now, we can use this function as a prop to the `useQuery` hook.

```
1 //src/App.tsx
2 import { useEffect, useState } from "react";
3 import { api } from "../api";
4 import { useQuery } from "@tanstack/react-query";
5 import { fetchUsers } from "../queryFunctions";
6
7 function App() {
8   // types for the data
9   type dataType = {
10     id: number;
11     name: string;
12     email: string;
13     password: string;
14   };
15
16   const { data, error, isLoading, refetch, isRefetching } = useQuery({
17     queryKey: ["users"],
18     queryFn: fetchUsers,
19   });
20
21   // checking the states
22   if (isLoading) return <div>Loading...</div>;
23
24   if (error) return <div>Error: {error.message}</div>;
25
26   if (isRefetching) return <div>Re-fetching...</div>;
27
28   if (!data) return <div>No data</div>;
29
30   //finally return the data
31   return (
32     <div>
33       {data.map((user) => (
34         <div key={user.id}>
35           <h2>{user.name}</h2>
36           <p>{user.email}</p>
37         </div>
38       ))}
39       <button onClick={() => refetch()}>Re-fetch</button>
40     </div>
41   );
42 }
43
44 export default App;
```

This final code will work exactly the same as the previous one. The only difference is that we're passing the query function as a prop to the useQuery hook.

One last thing is that I have to add some more `users` in the db.json file.

```
{
  "users": [
    {
      "id": 1,
      "name": "John Doe",
      "email": "2B7Pd@example.com",
      "password": "password123"
    },
    {
      "id": 2,
      "name": "Jane Doe",
      "email": "2B7Pd@example.com",
      "password": "password123"
    },
    {
      "id": 3,
      "name": "Jack Die",
      "email": "2B7Pd@example.com",
      "password": "password123"
    },
    {
      "id": 4,
      "name": "Bob dope",
      "email": "2B7Pd@example.com",
      "password": "password123"
    }
  ]
}
```

And we can now learn more about the `useQuery` hook.

Multiple QueryKeys

One thing that is very common in fetching data is getting that data according to a certain ID. For example, each of the users has an ID.

We can fetch a single user information from `/users/:id` endpoint.

Json-server pre-configures the endpoints to add dynamic routes by getting the id of the objects in the array.

Go ahead, type `http://localhost:3000/users/1` in your browser and you'll see the user information with Id 1.

But what about fetching them using `useQuery` hook?

We can follow, the same steps as we did in the first example.

let's make a query function is the `queryFunctions.ts` file.

```
1 import { api } from "../api";
2 export type dataType = {
3   id: number;
4   name: string;
5   email: string;
6   password: string;
7 };
8
9 export const fetchUsers = async () => {
10   const res = await api.get<dataType[]>("/users");
11   return res.data;
12 };
13
14 export const fetchUserById = async (id: number) => {
15   const res = await api.get<dataType>(`/users/${id}`);
16   return res.data;
17 };
```

I have added the `dataType` type to the `fetchUserById` function because I'm working in typescript.

Now, let's setup a new component for this example. Make a new folder in the `src` folder named `components`.

Now, make a new component file named `RenderUser.tsx` in the `components` folder.

```

1  //src/components/RenderUser.tsx
2  import React, { useState } from "react";
3
4  const RenderUser = () => {
5      const [id, setId] = useState(1);
6
7      return (
8          <div>
9              <input
10                 type="number"
11                 value={id}
12                 onChange={(e) => setId(parseInt(e.target.value))}
13             />
14             <button>Get User</button>
15         </div>
16     );
17 };
18
19 export default RenderUser;

```

Here, I have setup a simple input field to get the user ID and a button to fetch the user.

So, now we have to add it to the `App` component.

```

1  //src/App.tsx
2  import RenderUser from "../components/RenderUser";
3
4  function App() {
5      return (
6          <div>
7              <RenderUser />
8          </div>
9      );
10 }
11
12 export default App;

```

I've commented out almost everything in the `App` component from example 1 because I want to keep it clean for this example and now you should see a input field and a button rendered in the browser screen.

From the first example, we know how to use the `useQuery` hook to fetch data.

Now, What I want to do is enter a ID in the input field and then click the button to fetch the user.

Now, we know how the `useQuery` hook works. Let's try it like the first example.

```
1  //src/components/RenderUser.tsx
2  import { useQuery } from "@tanstack/react-query";
3  import React, { useState } from "react";
4  import { fetchUserById } from "../queryFunctions";
5
6  const RenderUser = () => {
7    const [id, setId] = useState(1);
8
9    const { data, isPending, error } = useQuery({
10      queryKey: ["user"],
11      queryFn: () => fetchUserById(id),
12    });
13
14    return (
15      <div>
16        {isPending && <h1>Loading...</h1>}
17        {error && <h1>{error.message}</h1>}
18        {data && (
19          <div>
20            <h1>{data.name}</h1>
21            <h1>{data.email}</h1>
22            <h1>{data.password}</h1>
23          </div>
24        )}
25        <input
26          type="number"
27          value={id}
28          onChange={(e) => setId(parseInt(e.target.value))}
29        />
30        <button>Get User</button>
31      </div>
32    );
33  };
34
35  export default RenderUser;
```

I'm checking the conditional states inside the `RenderUser` component because I don't want to break the UI or hide the input field whenever an error occurs. I want the input field to remain rendered at all times.

In the original code, I passed `id` as a parameter to `fetchUserById(id)`, expecting that updating the input would automatically run the fetch function again.

If you test that setup, it works perfectly on the first load for User 1. However, changing the ID input afterward completely fails to update the user data on the screen.

Why? Because the `useQuery` hook only activates automatically when its `queryKey` changes.

As a component mounts, the `useQuery` hook activates automatically. After that, it will only re-fetch if a value inside the `queryKey` changes (or due to background triggers like window refocus).

TanStack Query has no idea the dependencies changed. To fix this, the state variable must be included in the key.

This way every time the input value changes, the `queryKey` will change and the `useQuery` hook will re-run.

So, simply just add `["user", id]` to the `queryKey` array and it should work.

Nice right?

Now, this is not what I wanted. I wanted to fetch the user when the button is clicked instead of when the input value changes.

How, do we do that?

We can add a `enabled` prop to the `useQuery` hook to stop it from automatically running.

```
1 //src/components/RenderUser.tsx
2 const { data, isPending, error, refetch } = useQuery({
3   queryKey: ["user", id],
4   queryFn: () => fetchUserById(id),
5   enabled: false,
6 });
```

This will stop the `useQuery` hook from automatically running.

But the problem the query will never run if we don't use the `refetch` function. For now we can comment it out.

Now, we laid the groundwork that whenever `id` state will change the `useQuery` hook will re-run.

So, first we need to stop that. This might seem a little counterintuitive but everything will eventually work out. Trust me on this.

If we keep `id` inside both our typing input and the `queryKey`, TanStack Query gets confused the moment a user types. As you type a new number, the `queryKey` changes instantly. This resets the query status back to `isPending` and wipes your current data off the screen before you even get a chance to press the button!

To fix this, we need to completely `separate` the id we are currently typing from the id we actually want to fetch.

We do this by introducing a second state variable: `fetchedImage`.

```
1 // 1. This state tracks what the user is typing
2 const [id, setId] = useState(1);
3 // 2. This state ONLY updates when the button is actually clicked
4 const [fetchedImage, setFetchedImage] = useState<number | null>(null);
```

Now, instead of hooking `useQuery` up to our frantic typing input, we point it directly at `fetchedImage`.

```
1 const { data, isPending, error } = useQuery({
2   // Point the key and function to our submission state
3   queryKey: ["user", fetchedImage],
4   queryFn: () => fetchUserById(fetchedImage!),
5
6   // Magic line: Stay idle until the user actually clicks the button for the first time
7   enabled: fetchedImage !== null,
8 });
```

Look at how beautifully this solves all our problems at once:

1. **No accidental triggers:** Typing inside the input box updates `id`, but leaves `fetchedImage` completely alone. The UI stays perfectly stable while typing.
2. **Strict button control:** The fetch only activates inside our button's `handleGetUser` function, where we explicitly call `setFetchedImage(id)`.
3. **Perfect caching:** Because `fetchedImage` is in the `queryKey`, TanStack Query will cache every user individually (`["user", 1]`, `["user", 2]`). If you search for User 1, switch to User 2, and then search for User 1 again, it loads instantly from the cache without hitting your API a second time!

By decoupling our UI state from our network trigger state, we get the absolute best of both worlds.

And now, we can just add a `handle click` function to our button that will check if the `id` is valid or not. If it is, it will set the `fetchedImage` state to the `id` value and when the `fetchedImage` is not null, the query will run.

```
1 const handleGetUser = () => {
2   if (!isNaN(id)) {
3     setFetchedImage(id);
4   }
5 };
```

And now the only task is to bring this all together.

```
1 //src/components/RenderUser.tsx
2 import { useQuery } from "@tanstack/react-query";
3 import React, { useState } from "react";
4 import { fetchUserById } from "../queryFunctions";
5
6 const RenderUser = () => {
7   const [id, setId] = useState(1);
8   const [fetchedId, setFetchedId] = useState<number | null>(null);
9
10  const { data, isPending, error } = useQuery({
11    queryKey: ["user", fetchedId],
12    queryFn: () => fetchUserById(fetchedId!),
13    enabled: fetchedId !== null,
14  });
15
16  const handleGetUser = () => {
17    if (!isNaN(id)) {
18      setFetchedId(id);
19    }
20  };
21
22  return (
23    <div>
24      {isPending && <h1>Loading...</h1>}
25      {error && <h1>{error.message}</h1>}
26      {data && (
27        <div>
28          <h1>{data.name}</h1>
29          <h1>{data.email}</h1>
30          <h1>{data.password}</h1>
31        </div>
32      )}
33      <input
34        type="number"
35        value={id}
36        onChange={ (e) => setId(parseInt(e.target.value)) }
37      />
38      <button onClick={ () => handleGetUser() }>Get User</button>
39    </div>
40  );
41 };
42
43 export default RenderUser;
```

Note: The `handleGetUser` function is a simple function that checks if the `id` is a number, and if it is, it sets the `fetchId` state to the `id` value.

Everything should work how we wanted. The user data should only load when we click the button.

Only one thing left to do. A simple bug, you should see a `loading...` text rendered on the screen even though nothing is loading all the time.

This is because of the `isPending` state. It is a boolean state that is set to true when the query is loading, and false when it is done.

In this case you can use `isFetching` instead of `isPending` which is a boolean state that is by default set to false. But In any state, if the query is fetching at any time (including background refetching) `isFetching` will be true.

Note: Is pending is related to the state of the `queryFn` function. It is only false if the `queryFn` is in error or success state.

Now, there are a lot of other properties we can pass in the `useQuery` object. And those can be very useful for different situations.

The above example is for `dependent queries` which are queries that depend on the results of another query.

There can be multiple queries that depend on the results of another query.

Or Multiple queries that has to be fetched at the same time.

Let's talk more about these.

Query Options

By now I think you understand that the object passed inside the `useQuery` is the most important thing.

This is called a `Query options object`.

```
1 | const { data, isPending, error } = useQuery({  
2 |   queryKey: ["user", fetchedId],  
3 |   queryFn: () => fetchUserById(fetchedId!),  
4 |   enabled: fetchedId !== null,  
5 | });
```

This query can be written separately as:

```

1  const fetchIdQueryOption = {
2    queryKey: ["user", fetchedId],
3    queryFn: () => fetchUserById(fetchedId!),
4    enabled: fetchedId !== null,
5  };
6
7  const { data, isPending, error, isFetching } = useQuery(fetchIdQueryOption);

```

Because in the end it's just a simple object. You can export the object from other files and use it here if you want, it'll work the same.

Another better way to write this is using the `queryOptions` function.

```

1  const fetchIdQueryOption = queryOptions({
2    queryKey: ["user", fetchedId],
3    queryFn: () => fetchUserById(fetchedId!),
4    enabled: fetchedId !== null,
5  });
6
7  const { ... } = useQuery(fetchIdQueryOption);

```

Whats the benifit of using this?

Well, not much but it makes the code a little bit more readable and if you are using `typescript` then you can get type checking and property references. With plain object you won't get any of that.

Now, one more thing you can do is re-use the `queryOptions` object in multiple places.

Let's say you move the `queryOptions` object to the `queryFunctions` file.

```

1  //src/queryFunctions.ts
2  const fetchIdQueryOption = queryOptions({
3    queryKey: ["user", fetchedId],
4    queryFn: () => fetchUserById(fetchedId!),
5    enabled: fetchedId !== null,
6  });

```

Naturally this won't work because the `queryOptions` is taking `fetchId` state as an argument but the `fetchId` state is not in the `queryFunctions` file.

So, you can turn the `fetchIdQueryOption` object into a function that returns the `queryOptions` object.

```

1  //src/queryFunctions.ts
2  const fetchIdQueryOption = (fetchedId: number) =>
3    queryOptions({
4      queryKey: ["user", fetchedId],
5      queryFn: () => fetchUserById(fetchedId!),
6      enabled: fetchedId !== null,
7    });

```

and now you can import and pass the `fetchedId` state to the `fetchIdQueryOption` function.

```

1  //src/components/RenderUser.tsx
2  const RenderUser = () => {
3    const [id, setId] = useState(1);
4    const [fetchedId, setFetchedId] = useState<number | null>(null);
5
6
7    const { data, isPending, error, isFetching } = useQuery(fetchIdQueryOption(fetchedId));
8
9    ...
10
11 }

```

As you can see, instead of passing the `fetchIdQueryOption` object to the `useQuery` hook, you pass the `fetchIdQueryOption` **FUNCTION** to the `useQuery` hook and inside this function you pass the `fetchedId` state and everything works just fine.

Suspense Query

Exactly the same as `useQuery` but with the `guarantee` that the data will be available.

You can write it like this:

```

1  const { data, isPending, error } = useSuspenseQuery({
2    queryKey: ["user", fetchedId],
3    queryFn: () => fetchUserById(fetchedId!),
4    enabled: fetchedId !== null,
5  });

```

You know that `useQuery` hook can return the data as `unavailable` because it is async. But with the `useSuspenseQuery` hook will wait for the data to be available before rendering the component.

This hook works with the `react suspense` feature. You have to wrap the component with the component in the `App` component.

You should see that the UI is now waiting for the data to be available before rendering the component.

`useSuspenseQuery` hook completely ignores the `enabled` prop.

So, in this case the component will not work if you use `useSuspenseQuery` instead of `useQuery`.

Enabled prop stops the `useQuery` hook to fetch the data if the ID is null. But now as either way `useSuspenseQuery` will fetch the data, it'll show an error because there is no `/users/null` endpoint.

Queries

Let's say you have a component where multiple query has to be done simultaneously.

You can use the `useQueries` hook to do that.

Everything stays the same.

```
1  const queryOptions1 = queryOptions({
2    queryKey: ...,
3    queryFn: ...,
4    enabled: ...,
5  })
6
7  const queryOptions2 = queryOptions({
8    queryKey: ...,
9    queryFn: ...,
10   enabled: ...,
11  })
```

You can define these re-usable query options objects and you can pass these as a array to the `useQueries` hook.

```
1  const UseQueriesExample = useQueries({
2    queries: [queryOptions1, queryOptions2],
3  });
```

Note: The `useQueries` will take an object and inside that object you can pass a property called `queries` which is an array of query options objects.

The `useQueries` hook will return an array of objects. Each object will have the results for each query passed in the `queries` array.

```
1 | const [result1, result2] = useQueries({
2 |   queries: [queryOptions1, queryOptions2],
3 | });
```

If you want you can destructure the results from the array directly.

```
1 | const [
2 |   { data: data1, isPending: isPending1, error: error1 },
3 |   { data: data2, isPending: isPending2, error: error2 },
4 | ] = useQueries({
5 |   queries: [queryOptions1, queryOptions2],
6 | });
```

And that's it. For the very basics. You can also use `useSuspenseQueries` for multiple suspense queries and they will work the same way as `useSuspenseQuery` hook. Try it out yourself.

Stale And Fresh Data

Okay, Time to dig deeper.

You guys already know that you have to pass the `queryKey` to the `useQuery` hook.

Let's say you have two different `useQuery` hooks with `["users"]` and `["posts"]` query key.

When these queries are activated, TanStack uses these keys to keep track of the `data`. This is called `caching`.

Now, whenever you use the same queries again, the data will be fetched from the cache first, and after the data is fetched, the `data state` will update.

Now, how do you define which stays in cache and re-used, for how long?

Stale Time

In TanStack Query, you can define the `staleTime` option for each query.

This state time defines if the component should use the cached data completely or use the `queryFn` to fetch the data.

Stale time is a number in milliseconds to define how long the data should be cached.

```
1 | queryOptions({
2 |   queryKey: ...,
3 |   queryFn: ...,
4 |   staleTime: 60000 // 1 minute
5 | })
```

Here, I'm setting staledtime to 1 minute by passing 60000 milliseconds which 60 seconds.

This tells, tanstack query to use the cached data for 1 minute. And in that 1 minute if the query is re-run, tanstack will not use the queryFn to do network requests.

Let's try it out.

I'll setup a new component in the `components` folder named `StaleFresh.tsx`.

```
1 | //src/components/StaleFresh.tsx
2 |
3 | import React from "react";
4 | import { fetchUsers } from "../queryFunctions";
5 | import { useQuery } from "@tanstack/react-query";
6 |
7 | const StaleFresh = () => {
8 |   const { data } = useQuery({
9 |     queryKey: ["users"],
10 |    queryFn: fetchUsers,
11 |    staleTime: 60000,
12 |  });
13 |   return (
14 |     <div>
15 |       This is the data
16 |       {JSON.stringify(data)}
17 |     </div>
18 |   );
19 | };
20 |
21 | export default StaleFresh;
```

This component will send a request to the backend to get the data and as I've set the staleTime to 1 minute it'll keep the data fresh for 1 minute.

Now, I'll setup a toggle button to mount and unmount the component.

```

1  ...
2  function App() {
3
4      const [toggle, setToggle] = useState(true);
5      return (
6          <div>
7              {toggle && <StaleFersh />}
8              <button onClick={() => setToggle(!toggle)}>Toggle</button>
9          </div>
10     );
11 }
12
13 export default App;

```

Now, got to open you browser and go to the inspect tools network tab. Select the `fetch/XHR` tab and click the button to toggle the component. You'll see that the data is being re-fetched from cache every time the component is mounted but no network requests are being made.

You can also confirm that the data in cache is fresh by using the dev tools.

You can wait 1 min and toggle the component again and you'll see that the data is being re-fetched but this time the network request is being made.

So,

Stale time defines how long cached data of a query should be kept fresh.

Fresh data means, when A data is fresh the queryFn of that query will not be called.

When the staleTime is over, the cached data becomes stale and the queryFn will be called to fetch the data. And the cycle continues...

One nice thing is, If you are fetching some static data from backend that will never change you can set the `staleTime to Infinity`.

```

1  const { data } = useQuery({
2      queryKey: ["users"],
3      queryFn: fetchUsers,
4      staleTime: Infinity,
5  });

```

This will keep ur data fresh forever and will fecth data only at start up but never again after that.

Now, this bring me to another scenario.

Query invalidation

I think everyone has seen this in their react app.

You make a request to fetch data or delete data and you think that the UI will update accordingly.

But nothing happens to the UI until you refresh the page. This is because a request is not connected to the UI. It is a database operation.

So, whatever happens in the database the current UI should not be affected by it, that's the correct behavior right?

Same goes for tanstack query.

Let's say you have a function named `deleteUser` that deletes a user from the database.

```
1 | const deleteUser = (id) => {  
2 |   await axios.delete(`/users/${id}`);  
3 | };
```

Now, you have a component that shows a list of users.

```
1 | const Component = () => {  
2 |   const { data } = useQuery({  
3 |     queryKey: ["users"],  
4 |     queryFn: fetchUsers,  
5 |   });  
6 |  
7 |   return (  
8 |     <ul>  
9 |       {data.map((user) => (  
10 |         <li key={user.id}>  
11 |           {user.name} - {user.email}  
12 |           <button onClick={() => deleteUser(user.id)}>Delete</button>  
13 |         </li>  
14 |       ) )}  
15 |     </ul>  
16 |   );  
17 | };
```

Here, each user is rendered with a delete button that calls the `deleteUser` function what should delete the user from the database.

This will not update the UI until you call the `refetch` function for the `useQuery` hook.

So, we can use the `refetch` function to update the UI but that is not the correct way to do it.

The correct way to do it is to use the `invalidateQueries` function.

```
1  import { useQuery, useQueryClient } from "@tanstack/react-query";
2
3  const Component = () => {
4    const { data } = useQuery({
5      queryKey: ["users"],
6      queryFn: fetchUsers,
7    });
8
9    const queryClient = useQueryClient();
10
11    const handleDelete = async (id) => {
12      await deleteUser(id);
13      queryClient.invalidateQueries({
14        queryKey: ["users"],
15      });
16    };
17
18    return (
19      <ul>
20        {data.map((user) => (
21          <li key={user.id}>
22            {user.name} - {user.email}
23            <button onClick={() => handleDelete(user.id)}>Delete</button>
24          </li>
25        ))}
26      </ul>
27    );
28  };
```

`InvalidateQueries` is method of the `queryClient` object. That's we have to use the `useQueryClient` hook to get the `queryClient` object.

And I made a function called `handleDelete` that calls the `deleteUser` function and then calls the `invalidateQueries` function to invalidate the query.

`invalidateQueries` takes an object where you must pass the `queryKey` which is an array of strings that will be used to identify the query and that query will be invalidated and refetched through the network.

Now, let's understand why I'm using the `invalidateQueries` function instead of the `refetch` function.

1. Invalidation is Smart (Respects staleTime), refetch() is Brutalrefetch() forces a network request immediately, no matter what.

- It completely `ignores` your `staleTime` configurations. Even if you fetched the exact same data one second ago and it is perfectly "fresh," `refetch()` will blindly hit your server again.
- `invalidateQueries()` marks the data as "stale" (expired) first. It tells TanStack Query: "Hey, the data for this key is now old and dirty."
 - If the component using that query is currently visible on the screen, TanStack Query will instantly `refetch` it in the background.
 - If the component is not on the screen (e.g., it's on a different page), TanStack Query will not fetch it right away. It will wait until the user navigates back to that page before pulling the fresh data. This saves massive amounts of server bandwidth.

2. Invalidation scale across your entire app (The Network Effect)

- Imagine you have a dashboard. You have a `UserProfile` component at the top of the screen, and a `SettingsForm` component at the bottom of the screen. Both rely on the `["user", id]` data.
- If you change the user's name in the `SettingsForm` and call a manual `refetch()` inside that form, only that specific instance of the hook `refetches`. The `UserProfile` component at the top of the screen might stay stuck showing the old name.
- If you use `queryClient.invalidateQueries({ queryKey: ["user", id] })`, TanStack Query `broadcasts` a message to the entire application. Every single component on the screen that cares about `["user", id]` will simultaneously `refresh` itself to show the brand-new data.

3. Invalidation works seamlessly with `Mutations`. I'll talk more about mutations right after this.

One very minute details that I almost forgot to talk about is that the `invalidateQueries` function is also returning a promise. This means that `invalidateQueries` is a asynchronous function and some time might take to invalidate the query.

So, if you don't use `await` before calling the `invalidateQueries` function and do some operation connected to the query that is being invalidated, the operation will be executed on the OLD data, not the newly updated data.

Now, let's talk about Mutations.

Mutations

While `useQuery` is for fetching data, more accurately it is for `get` requests, `useMutation` is for `post`, `put`, `delete` requests.

In the above example, I made a function named `deleteUser` that deletes a user from the database.

And I'm using it raw without any try catch block and or any kind of success and error handling.

`Mutation` will give us all that but with some more powerful features.

`useMutation` hook works almost the same way as `useQuery` hook.

```
1 | import { useMutation } from "@tanstack/react-query";
2 |
3 | const Component = () => {
4 |   const { mutate } = useMutation({
5 |     mutationFn: (id) => {
6 |       return someRequest(id);
7 |     },
8 |   });
9 | };
```

Now, let's understand the structure. First the use `useMutation` hook takes an object just like the `useQuery` hook.

Inside that function you can pass a property called `mutationFn` which is a function that will be used to make the request.

If the function takes any arguments, you can get the arguments from the `useMutation` hook and then pass them in the request function with a callback.

Now, the main thing here is what you will get in return from the hook. It'll return a function called `mutate`. You'll use this function to activate the request.

So, let's turn the previous example into a mutation.

```

1 //src/components/Mutations.tsx
2 ...
3
4 const Mutations = () => {
5   ...
6
7   const {mutate} = useMutation({
8     mutationFn: (id: number) => deleteUser(id),
9   })
10
11   const handleDelete = (id: number) => {
12     mutate(id)
13     client.invalidateQueries({
14       queryKey: ["users"],
15     });
16   };
17
18   ...

```

I made a new component named `Mutations` in the `components` folder and copied and pasted the code from the previous example.

Now, instead of directly calling the `deleteUser` function, we call the `mutate` function inside the `handleDelete` function.

Now, you might ask what's the point. We have an extra like and nothing changed everything should still work right?

Nope, you'll see the same issue as before. The UI is not updating even after the mutation function is called.

Why?

Because the `mutate` activated the request but it is being handled in an asynchronous process and invalidation is being handled in another asynchronous process.

So, while the invalidation is done, the request is resolved and the UI is not updated because the mutation function is not done deleting the user.

So, we need a way to put the two together.

And in the mutation function you can actually do that by passing another property called `onSuccess` to the `useMutation` hook.

This property will activate only if the request of the `mutate` function is resolved.

```
1 //src/components/Mutations.tsx
2 ...
3
4 const Mutations = () => {
5   ...
6
7   const {mutate} = useMutation({
8     mutationFn: (id: number) => deleteUser(id),
9     onSuccess: () => {
10       client.invalidateQueries({
11         queryKey: ["users"],
12       });
13     }
14   })
15
16   const handleDelete = (id: number) => {
17     mutate(id)
18   };
19
20   ...
```

Now, if you call the `handleDelete` function, the UI will update because the `onSuccess` function will only activate if the delete request is resolved.

you can also pass the `onError` property to the `useMutation` hook to tell it what to do if the request fails.

Or `onSettled` property to tell it what to do after the request is successful or failed. Either way this function will activate.

If you go to the official [docs](#) you can see the full list of properties that you can pass to the `useMutation` hook.

tsx



```
useMutation({
  mutationFn: addTodo,
  onMutate: (variables, context) => {
    // A mutation is about to happen!

    // Optionally return a result containing data to use when for example rolling back
    return { id: 1 }
  },
  onError: (error, variables, onMutateResult, context) => {
    // An error happened!
    console.log(`rolling back optimistic update with id ${onMutateResult.id}`)
  },
  onSuccess: (data, variables, onMutateResult, context) => {
    // Boom baby!
  },
  onSettled: (data, error, variables, onMutateResult, context) => {
    // Error or success... doesn't matter!
  },
})
```

Here, you can see that each property is function that give you arguments like

`data`, `error`, `onMutateResults`, `variables` and `context`.

- `data` is the data that is returned from the request.
- `error` is the error that is returned from the request.
- `onMutate` is another sideEffect function of the `mutate` function. It executes when the `mutate` function is called. If you want to do something extra while the `mutate` function is called, you can do it here and you can return something from it.
- `variables` is the arguments that are passed to the mutation function.
- `onMutateResult` is the returned value of the `onMutate` function.
- `context` is the metadata object of the mutation.

There are other query methods and hooks like `useInfiniteQuery`, `useQueryClient` and `useSuspenseQuery` that also have similar properties.

But this article was to educated about the main three concepts of `tanstack query`.

Last words

This took longer than I expected. ciao.